

Conditional Deep Convolutional GAN

(CDCGAN) 구현 보고서

MNIST 조건부 이미지 생성 모델 설계 및 분석 보고서

AI빅데이터융합경영학과

20212550 김태현

1. 서론

본 보고서는 MNIST 데이터셋을 대상으로 Conditional Deep Convolutional GAN (CDCGAN)을 설계하고 학습한 과정과 결과를 기술한다. CDCGAN은 기존 DCGAN(Deep Convolutional GAN)에 클래스 조건(class label)을 추가하여, 사용자가 원하는 숫자(0~9)를 지정하면 해당 숫자에 해당하는 이미지를 생성할 수 있도록 확장한 생성 모델이다.

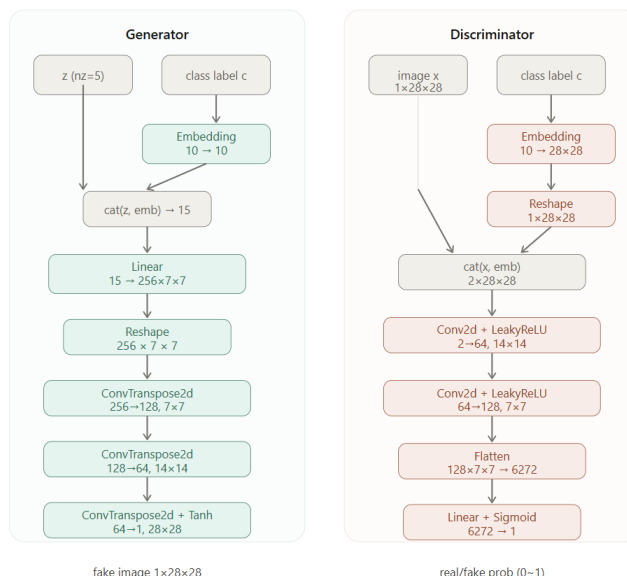
기존 DCGAN은 랜덤 노이즈 벡터 z 만을 입력으로 받아 이미지를 생성하기 때문에, 어떤 숫자가 생성될지 제어할 수 없다. 반면 CDCGAN은 노이즈 벡터 z 와 함께 클래스 레이블 c 를 조건으로 입력받아 원하는 숫자를 지정하여 생성할 수 있다. 이것이 "Conditional"의 의미이며, 본 과제 핵심 구현 목표이다.

2. 모델 설계

2.1 DCGAN에서 CDCGAN으로의 확장 방법

CDCGAN은 제공된 DCGAN 코드(dcgan.zip)를 기반으로, 생성자와 판별자 모두에 클래스 조건 c 를 추가하는 방식으로 확장하였다. 핵심 아이디어는 클래스 레이블을 Embedding 레이어를 통해 연속적인 벡터로 변환하고, 이를 기존 입력에 결합(concat)하는 것이다.

생성자에서만 c 를 사용하고 판별자에서는 사용하지 않는다면, 판별자가 클래스 조건을 인식하지 못해 생성자가 조건을 무시하고 임의의 숫자를 생성하더라도 판별자가 이를 잡아내지 못한다. 따라서 생성자와 판별자 양쪽 모두에 c 를 적용하는 것이 올바른 CDCGAN 구조이다.



2.2 조건부 생성자 (CGenerator)

조건부 생성자는 DCGAN의 Generator를 확장하여 클래스 조건 c 를 함께 입력받도록 수정하였다. 구체적인 확장 방법은 다음과 같다.

- 클래스 레이블 c 를 $\text{Embedding}(10, 10)$ 레이어를 통해 크기 10의 임베딩 벡터로 변환한다.
- 노이즈 벡터 z (크기 5)와 임베딩 벡터(크기 10)를 torch.cat 으로 결합하여 크기 15의 벡터를 만든다.
- 이후 DCGAN과 동일하게 $\text{Linear} \rightarrow \text{Reshape} \rightarrow \text{ConvTranspose2d}$ 레이어를 통해 28×28 이미지를 생성한다.
- DCGAN의 $\text{forward}(z)$ 를 $\text{forward}(z, c)$ 로 변경하여 클래스 조건을 함께 처리한다.

DCGAN과의 가장 큰 차이점은 첫 번째 Linear 레이어의 입력 크기이다. DCGAN은 $n_z(=100)$ 를 입력받았으나, CDCGAN은 $n_z + \text{num_classes}(=5 + 10 = 15)$ 를 입력받도록 수정하였다.

레이어	출력 크기	파라미터	비고
$\text{Embedding}(10, 10)$	10	-	클래스 레이블 임베딩
$\text{cat}(z, \text{emb})$	$5 + 10 = 15$	-	DCGAN과의 핵심 차이
$\text{Linear} + \text{BN} + \text{LReLU}$	$256 \times 7 \times 7$	$\text{in}=15, \text{out}=12544$	Reshape 포함
$\text{ConvTranspose2d} + \text{BN} + \text{LReLU}$	$128 \times 7 \times 7$	$k=5, s=1, p=2$	-
$\text{ConvTranspose2d} + \text{BN} + \text{LReLU}$	$64 \times 14 \times 14$	$k=4, s=2, p=1$	-
$\text{ConvTranspose2d} + \text{Tanh}$	$1 \times 28 \times 28$	$k=4, s=2, p=1$	최종 출력 ($-1 \sim 1$)

- 표 1. CGenerator 레이어 구성 -

2.3 조건부 판별자 (CDiscriminator)

조건부 판별자는 DCGAN의 Discriminator를 확장하여 이미지 x 와 함께 클래스 조건 c 를 입력받도록 수정하였다. 생성자와 달리 판별자는 이미지와 동일한 공간 구조를 가져야 하기 때문에, 다른 방식으로 c 를 결합한다.

- 클래스 레이블 c 를 $\text{Embedding}(10, 28 \times 28)$ 레이어를 통해 784 차원 벡터로 변환한다.
- 이를 Rearrange 를 사용해 $1 \times 28 \times 28$ 크기의 공간적 채널로 reshape 한다.
- 이미지 $x(1 \times 28 \times 28)$ 와 임베딩 채널($1 \times 28 \times 28$)을 채널 방향으로 concat 하여 $2 \times 28 \times 28$ 입력을 구성한다.
- DCGAN의 첫 Conv2d 입력 채널을 1에서 2로 변경하여 결합된 입력을 처리한다.
- DCGAN의 $\text{forward}(x)$ 를 $\text{forward}(x, c)$ 로 변경하여 클래스 조건을 함께 처리한다.

이 방식은 판별자가 단순히 이미지가 진짜인지 가짜인지만 판별하는 것이 아니라, 해당 클래스 조건에 맞는 이미지인지 함께 판별할 수 있게 한다. 예를 들어 $c=5$ (숫자 5)가 주어졌을 때, 숫자 3의 이미지는 가짜로 판별하게 된다.

레이어	출력 크기	파라미터	비고
Embedding(10, 784) + Reshape	1×28×28	-	공간 임베딩
cat(x, emb)	2×28×28	-	DCGAN과의 핵심 차이
Conv2d + LReLU + Dropout	64×14×14	k=4, s=2, p=1	in_ch: 1→2
Conv2d + LReLU + Dropout	128×7×7	k=4, s=2, p=1	-
Flatten + Linear + Sigmoid	1	in=6272, out=1	진짜/가짜 확률

- 표 2. CDiscriminator 레이어 구성 -

2.4 CDCGAN Lightning 모듈

DCGAN의 Lightning 모듈(DCGAN 클래스)을 CDCGAN 클래스로 확장하였다. 주요 변경 사항은 다음과 같다.

- 클래스명: DCGAN → CDCGAN
- 생성자/판별자: Generator/Discriminator → CGenerator/CDiscriminator
- __init__ 인자에 num_classes=10 추가
- training_step 에서 $x, _ = \text{batch}$ 를 $x, c = \text{batch}$ 로 변경하여 클래스 레이블을 활용
- 생성자 호출: $\text{self}(z) \rightarrow \text{self}(z, c)$, 판별자 호출: $\text{discriminator}(x) \rightarrow \text{discriminator}(x, c)$

3. 학습

3.1 학습 환경 및 하이퍼 파라미터

학습은 Google Colab 환경에서 진행하였다. 하이퍼파라미터는 스켈레톤 코드에서 제공된 기본값을 그대로 사용하였으며, max_epochs=3으로 학습을 수행하였다. 무료 Colab 환경의 런타임 제약이 있었으나, 3에폭만으로도 클래스 조건이 명확하게 반영된 이미지가 생성되어 조건부 생성 모델로서의 동작을 검증하기에 충분하였다.

하이퍼 파라미터	설정값
데이터셋	MNIST (train: 60,000장 / test: 10,000장)
배치 크기 (batch_size)	100
노이즈 벡터 차원 (nz)	5
클래스 수 (num_classes)	10 (숫자 0~9)
에폭 수 (max_epochs)	3
학습률 (lr)	0.0002
옵티마이저	Adam (beta1=0.5, beta2=0.999)
손실 함수	Binary Cross Entropy (BCE)
학습 프레임워크	PyTorch Lightning
학습 환경	Google Colab (T4 GPU)

- 표 3. 학습 하이퍼파라미터 -

3.2 적대적 학습 과정 (Adversarial Training)

GAN 학습은 생성자(G)와 판별자(D)를 번갈아가며 학습하는 적대적(adversarial) 학습 방식을 사용한다. PyTorch Lightning의 수동 최적화(manual optimization) 모드를 활용하여 두 네트워크를 독립적으로 업데이트한다.

생성자 학습 단계에서는 랜덤 노이즈 z 와 클래스 레이블 c 로 가짜 이미지를 생성한 뒤, 판별자가 이를 진짜(label=1)로 판단하도록 생성자 파라미터를 업데이트한다. 손실 함수는 다음과 같다.

$$\text{loss_G} = \text{BCE}(\text{D}(\text{G}(z, c), c), 1)$$

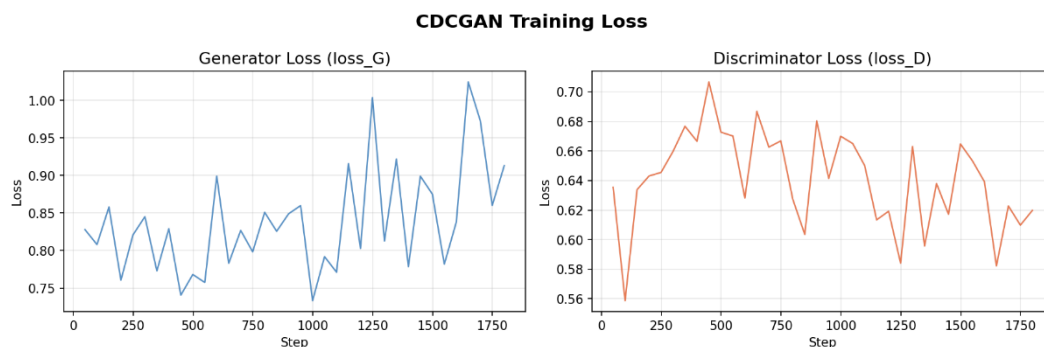
판별자 학습 단계에서는 진짜 이미지에 대해서는 1, 가짜 이미지에 대해서는 0으로 판별하도록 판별자 파라미터를 업데이트한다. 손실 함수는 다음과 같다.

$$\text{loss_D} = [\text{BCE}(\text{D}(x, c), 1) + \text{BCE}(\text{D}(\text{G}(z, c), c), 0)] / 2$$

두 네트워크가 경쟁하며 학습이 진행될수록 생성자는 더 그럴듯한 이미지를 생성하게 되고, 판별자는 더 정교하게 진짜와 가짜를 구별하게 된다. 이상적인 GAN 학습 수렴 상태에서 loss_D 는 약 $0.693(\ln 2)$ 에 수렴한다.

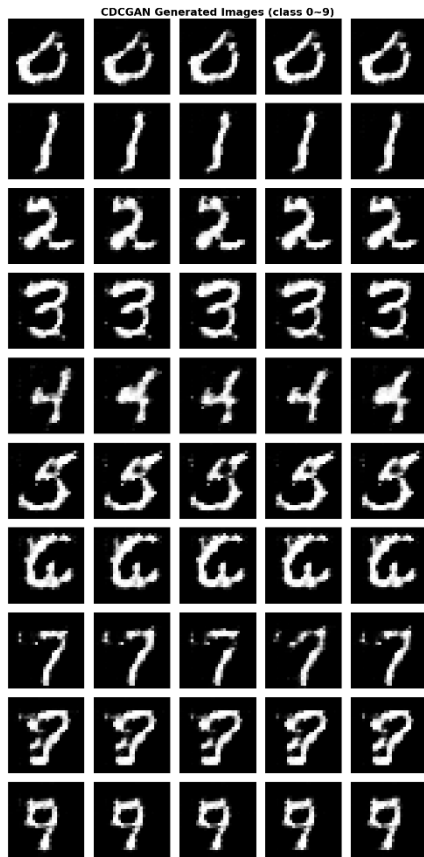
3.3 학습 손실 그래프

아래 그래프는 학습 step에 따른 Generator 손실(loss_G)과 Discriminator 손실(loss_D)의 변화를 나타낸다. 총 3에폭, 약 1800 step의 학습이 진행되었다.



loss_D 는 초반 약 0.56까지 낮아진 후 0.6~0.7 구간에서 진동하며 점차 안정되는 양상을 보인다. 이는 이론적 수렴값인 $\ln 2 (\approx 0.693)$ 에 근접한 값으로, 판별자가 진짜와 가짜를 완벽히 구분하지도 완전히 속지도 않는 균형 상태에 근접하고 있음을 의미한다. loss_G 는 0.75~1.0 구간에서 진동하며, 3에폭이라는 짧은 학습에도 불구하고 생성자가 판별자를 어느 정도 속이는 능력을 갖추었음을 나타낸다.

4. 결과 분석



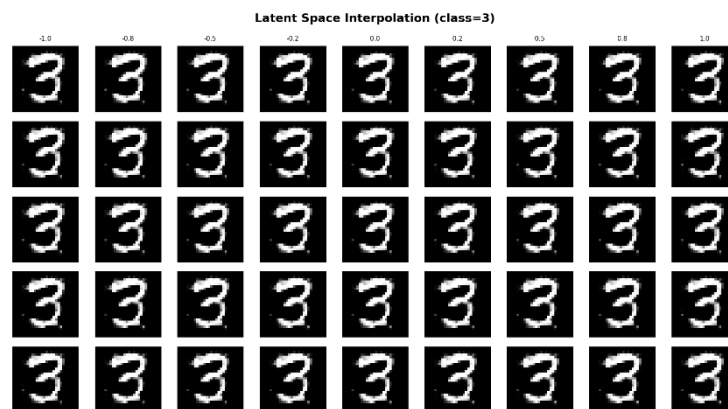
4.1 클래스별 생성 이미지

학습이 완료된 생성자에 각 클래스 레이블(0~9)과 서로 다른 노이즈 벡터 z 를 입력하여 각 클래스당 5장씩 총 50장의 이미지를 생성하였다.

생성 결과를 보면 0~9 각 클래스에 해당하는 숫자가 명확하게 생성됨을 확인할 수 있다. 특히 1, 2, 3, 7, 9와 같이 획의 구조가 단순한 숫자는 매우 선명하게 생성되었으며, 같은 클래스 내에서도 노이즈 벡터 z 에 따라 획의 굵기나 기울기 등 스타일이 다양하게 생성되어 모델이 단순히 동일한 이미지를 반복하지 않음을 확인할 수 있다. 3에폭이라는 짧은 학습임에도 불구하고 조건부 생성 모델로서 충분히 동작함을 보여 준다.

4.2 노이즈 벡터 보간 실험

잠재 공간(latent space)의 특성을 파악하기 위해, 클래스 $c=3$ 으로 고정한 상태에서 노이즈 벡터 z 의 각 차원($z_0 \sim z_4$)을 -1.0에서 1.0으로 변화시키며 생성 이미지의 변화를 관찰하였다.

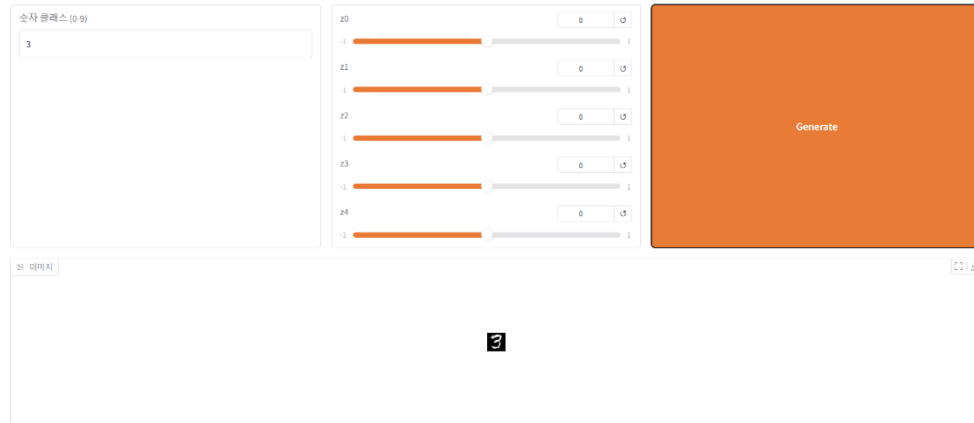


보간 실험 결과, z 값이 변화하더라도 숫자 3의 전반적인 형태는 일관되게 유지되었다. 각 차원별로 뚜렷하게 구분되는 시각적 변화는 관찰되지 않았는데, 이는 $n_z=5$ 로 잠재 공간의 차원이 매우 작아 각 차원이 독립적인 이미지 속성을 담당하기 어렵기 때문으로 분석된다. 그럼에도 클래스 조건 $c=3$ 이 모든 z 값에서 일관되게 유지된다는 점에서, 조건부 생성이 정상적으로 작동함을 확인할 수 있다. n_z 를 더 크게 설정하면 각 차원이 획의 굵기, 기울기, 위치 등 서로 다른 이미지 속성을 제어할 수 있을 것으로 기대된다.

4.3 실시간 생성 테스트 (Gradio UI)

gradio_test.py를 실행하여 학습된 CDCGAN 생성 네트워크를 웹 브라우저 기반의 인터랙티브 UI로 테스트하였다. 사용자는 숫자 클래스(0~9)와 노이즈 벡터의 각 차원($z_0 \sim z_4$)을 슬라이더로 직접 조작하여 실시간으로 이미지를 생성할 수 있다.

CDCGAN - 조건부 이미지 생성기



Gradio UI 테스트를 통해 클래스 레이블을 변경하면 즉시 해당 숫자에 대응하는 이미지가 생성됨을 확인하였다. 다만 노이즈 슬라이더($z_0 \sim z_4$)를 조작하였을 때 생성 이미지의 변화가 크지 않았는데, 이는 보간 실험에서 관찰된 것과 동일하게 $n_z=5$ 의 작은 잠재 공간 차원에 기인한 것으로 분석된다. 그럼에도 클래스 조건에 따른 이미지 생성이 실시간으로 정상 작동함을 검증하였다.

5. 결론 및 한계

본 과제를 통해 DCGAN 코드를 기반으로 CDCGAN을 성공적으로 구현하고 MNIST 데이터셋에서 학습하였다. DCGAN과 CDCGAN의 핵심적인 차이는 클래스 레이블 c 를 Embedding 레이어를 통해 벡터로 변환하고, 생성자와 판별자 양쪽 모두의 입력에 결합하는 것이다.

구현 과정에서 얻은 주요 결과와 분석은 다음과 같다.

- CGenerator: 클래스 임베딩(크기 10)을 노이즈 z (크기 5)와 concat 하여 조건부 이미지 생성 구현. 첫 Linear 레이어 입력 크기를 n_z 에서 $n_z + \text{num_classes}$ 로 수정하는 것이 핵심이었다.
- CDiscriminator: 클래스 임베딩을 이미지와 동일한 28×28 공간으로 변환하여 채널 방향으로 결합. 첫 Conv2d의 입력 채널을 1에서 2로 변경하였다.
- 클래스별 생성 이미지: 0~9 각 클래스에 대해 명확하게 구분되는 숫자 이미지가 생성되어, 조건부 생성 모델로서 정상 작동함을 확인하였다.
- 학습 손실: loss_D 가 0.6~0.7 구간에서 안정적으로 유지되어 생성자와 판별자가 균형 있게 학습되었음을 확인하였다.

다만 Colab 환경의 런타임 제약으로 인해 $\text{max_epochs}=3$ 으로 학습을 제한하였으며, 이로 인해 loss_G 가 완전히 수렴하지 못한 한계가 있다. 향후 예폭 수 증가, n_z 확장, 더 깊은 네트워크 구조 적용 등을 통해 생성 이미지의 품질을 더욱 향상시킬 수 있을 것으로 기대된다.